

COMPUTER NETWORKS MODULE - 1 ANSWERS

1. (a) OSI vs TCP/IP for troubleshooting

The **OSI model** is more effective for troubleshooting because it has **seven clearly defined layers**, which allows network problems to be isolated step-by-step, starting from the Physical layer up to the Application layer. This layered approach helps in identifying whether the issue is related to hardware, addressing, routing, transport, or application services.

The **TCP/IP model** has only **four layers**, making it simpler and more practical for real-world Internet communication, but it provides **less detailed separation** of functions. Because of this, it is not as precise as OSI for pinpointing the exact layer where a fault occurs.

In practice, the **OSI model is mainly used as a conceptual and troubleshooting framework**, while the **TCP/IP model is used for actual network implementation**.

(b) Web, DNS, E-mail and their application layer protocols

The **Web** service is used to access web pages and online resources. It works using the **HTTP** protocol and the secure version **HTTPS**, which enable communication between a web browser and a web server.

The **DNS (Domain Name System)** is used to translate human-readable domain names into IP addresses. It uses the **DNS protocol** to perform name resolution, such as converting `www.google.com` into an IP address.

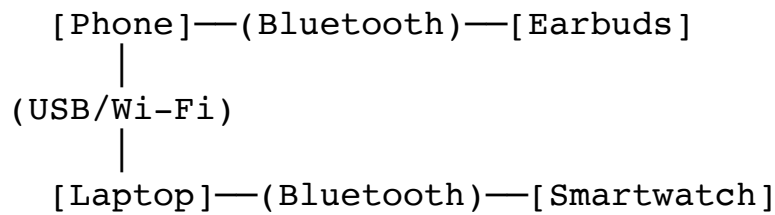
E-mail is used for sending and receiving electronic messages. The main protocols used are **SMTP** for sending emails, **POP3** for receiving and downloading emails from the server, and **IMAP** for receiving and synchronizing emails with the server.

(c) PAN (Personal Area Network)

A **PAN (Personal Area Network)** is a small network used to connect personal devices over a **short distance**, usually within a few meters around a person. It is mainly designed for **personal use** and supports communication between devices such as mobile phones, laptops, smartwatches, wireless earbuds, and printers.

Common technologies used in PAN include **Bluetooth**, **USB**, and short-range **Wi-Fi**. PANs are widely used for tasks like transferring files between devices, connecting wireless earphones to a phone, or linking a smartwatch to a smartphone.

Simple diagram (you can draw in exam):



2. (a) Types of network topologies used in a company

A **network topology** is the physical or logical layout of devices in a network. Common topologies used in companies are:

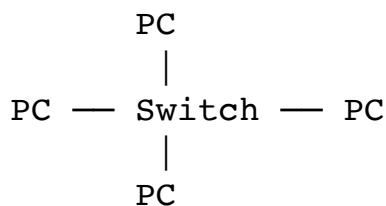
1. Bus Topology

- All devices share a single backbone cable.
- Simple and low cost, but a fault in the main cable can stop the whole network.

PC — PC — PC — PC

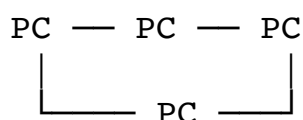
2. Star Topology

- All devices connect to a central switch or hub.
- Easy to manage and troubleshoot; widely used in offices.
- If one cable fails, only that device is affected.



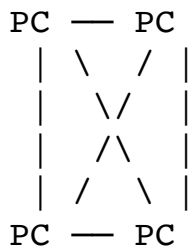
3. Ring Topology

- Each device is connected in a circular manner.
- Data travels in one direction around the ring.
- A break in the ring can disrupt communication.



4. Mesh Topology

- Each device is connected to many or all other devices.
- Very reliable and fault-tolerant, but expensive and complex.



5. Hybrid Topology

- Combination of two or more topologies (e.g., star + bus).
- Common in large companies and multinational offices.

(b) Causes of delay in the corporation's network

Delays in a company network, especially with **real-time applications and international offices**, can occur due to several reasons:

1. Geographical distance:

- Data traveling between faraway offices takes more time due to **signal propagation delay**.
- Longer physical distance = higher delay.

2. Network infrastructure capacity:

- Limited bandwidth, slow routers, or overloaded switches can cause **congestion**.
- When many users send data at the same time, packets must wait in queues.

3. Type of applications used:

- Real-time applications (video calls, VoIP, live streaming) need **low latency**.
- Heavy applications (cloud backups, file transfers) consume more bandwidth and increase delay.

4. Traffic load and routing:

- Poor routing paths or high traffic on certain links can increase transmission time and waiting time.

(c) Different types of delay in a network

In computer networks, total delay is mainly composed of the following:

1. Propagation Delay

- Time taken for a signal to travel from sender to receiver.
- Depends on **distance** and **signal speed** in the medium.
- Important in long-distance (international) communication.

2. Transmission Delay

- Time required to **push all bits of a packet** onto the link.
- Depends on **packet size** and **link bandwidth**.
- Larger data or lower bandwidth → more delay.

3. Queuing Delay

- Time a packet waits in a **router or switch queue** before being transmitted.
- Increases when the network is **congested**.

4. Processing Delay

- Time taken by routers/switches to **process the packet** (error checking, header reading, routing decision).
- Usually small, but adds up in large networks.

In summary:

Total network delay = Propagation delay + Transmission delay + Queuing delay + Processing delay.

Great—here's a **clear, exam-ready** answer set, structured, concise, and easy to reproduce in the paper.

3. (a) Advantages of Virtual Circuits and how segmentation improves performance and security

A **Virtual Circuit (VC)** is a logical, pre-established path between sender and receiver before data transfer begins.

Primary advantages:

- **Predictable performance:** Since the path is fixed, delay and throughput are more stable—good for real-time applications.
- **Efficient routing:** Routers do not need to make complex routing decisions for every packet after setup.
- **Better resource management:** Bandwidth and buffers can be reserved in advance, reducing congestion.
- **In-order delivery:** Packets arrive in the same order they are sent, reducing the need for reordering at the receiver.

How virtual circuit segmentation helps:

- **Performance:** Large messages are split into smaller packets (segments), which allows **pipelining** and better use of the link bandwidth. This reduces waiting time and improves throughput.
- **Security:** Segmentation limits exposure—if one packet is intercepted, it contains only a **small part of the data**. Also, since the path is predefined, it is easier to apply **access control, monitoring, and encryption** on that circuit.

(b) Virtual circuit networks vs Datagram networks

Feature	Virtual Circuit Network	Datagram Network
Connection	Connection-oriented (setup required)	Connectionless (no setup)
Path	Fixed path for all packets	Each packet may take a different path
Routing	Route decided once during setup	Route decided for every packet
Delay	More predictable	Can vary from packet to packet
Order of packets	Usually in order	May arrive out of order
Example	ATM, Frame Relay	IP (Internet)

In short:

- **Virtual circuits** focus on **reliability and predictability**.
- **Datagram networks** focus on **flexibility and scalability**.

(c) Types of errors, their impact, and how they are reduced

Common types of errors:

1. **Bit errors:** Bits change from 0 to 1 or 1 to 0 due to noise or interference.
2. **Packet loss:** Packets may be dropped due to congestion or buffer overflow.
3. **Packet corruption:** Data gets damaged during transmission.
4. **Packet duplication or reordering:** Same packet may arrive twice or in wrong order.

Impact on network performance and reliability:

- Causes **retransmissions**, increasing delay and reducing throughput.
- Degrades **quality of real-time applications** (voice/video).
- Reduces **overall reliability** and user experience.
- Increases **network load** due to repeated transmissions.

Error reduction using error control techniques:

- **Error detection:** Parity check, Checksum, CRC to detect errors.
- **Error correction:**
 - **ARQ (Automatic Repeat reQuest):** Retransmit lost or corrupted packets (e.g., Stop-and-Wait, Go-Back-N, Selective Repeat).
 - **Forward Error Correction (FEC):** Add redundant bits so receiver can correct some errors without retransmission.
- **Flow and congestion control:** Reduce overload, which lowers packet loss and errors.

Conclusion:

Using proper **error detection and correction techniques** improves both **performance** and **reliability** of the network by reducing data loss, retransmissions, and delays.

Perfect—here is your **combined, clean, exam-ready answer** for Question 4 with the **required diagrams included directly** and the content merged smoothly.

4. Secure Communication Using Transport Layer and TLS

(a) Using transport layer services to ensure secure and encrypted communication

To establish a secure communication channel between two geographically distant financial offices, the **transport layer** is used along with **encryption mechanisms** to protect sensitive financial data. The most common and effective approach is to use **TCP with TLS (Transport Layer Security)**.

First, **TCP (Transmission Control Protocol)** provides **reliable, ordered, and error-free** data transfer between the two locations. On top of TCP, **TLS** is applied to **encrypt the data** before transmission. During the connection setup, TLS performs **authentication** of the server (and optionally the client), **exchanges cryptographic keys**, and establishes a **secure session**.

Once the secure session is established:

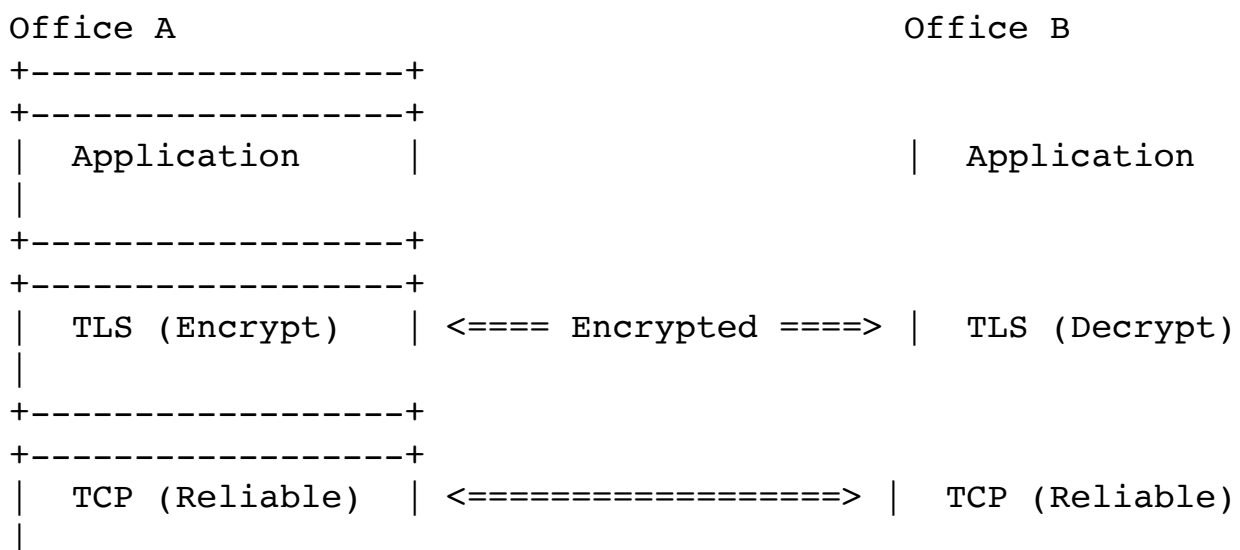
- All data is **encrypted** before sending.
- The receiver **decrypts** the data using the shared secret keys.
- Any **tampering** with the data can be **detected**.

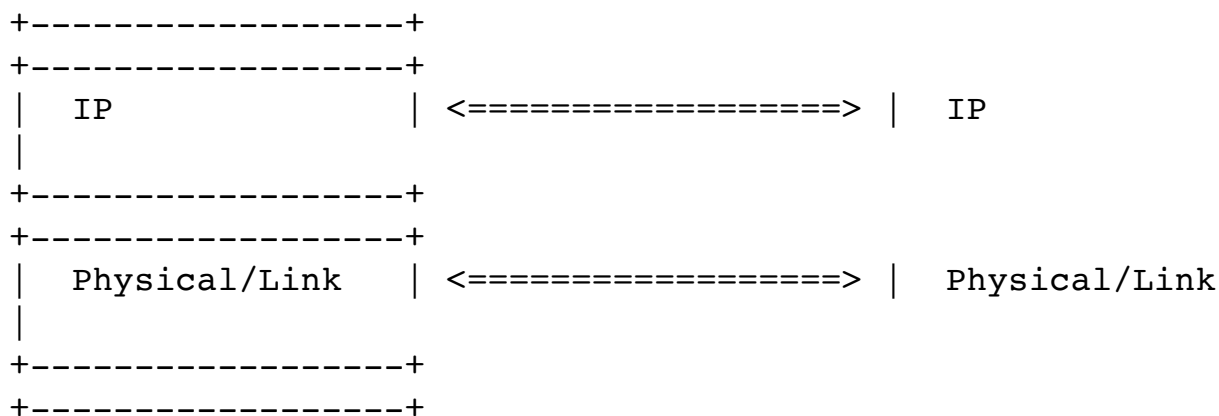
This ensures:

- **Confidentiality:** Unauthorized users cannot read the data.
- **Integrity:** Data is not altered during transmission.
- **Authentication:** Communication happens only between trusted endpoints.

Thus, by using **TCP + TLS**, secure and reliable communication of sensitive financial information can be achieved between distant offices.

Diagram: TCP + TLS Secure Communication





(b) Significance of transport layer protocols: TCP and UDP

The **transport layer** is responsible for **end-to-end communication** between applications running on different devices. The two main transport layer protocols are **TCP** and **UDP**.

TCP (Transmission Control Protocol):

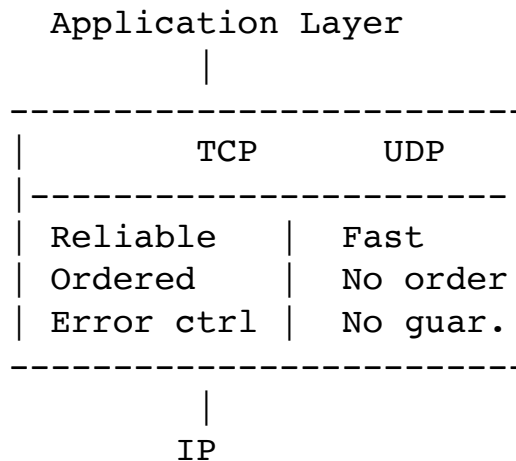
- **Connection-oriented** and reliable.
- Provides **error detection, retransmission, flow control, and congestion control**.
- Ensures **ordered and complete delivery** of data.
- Suitable for applications where **accuracy and reliability** are critical, such as:
 - Web browsing (HTTPS)
 - File transfer
 - Email
 - Financial transactions

UDP (User Datagram Protocol):

- **Connectionless** and faster, but **unreliable**.
- No guarantee of **delivery, order, or duplication**.
- Has **low overhead** and **low latency**.
- Suitable for **real-time applications** such as:
 - Video streaming
 - Voice over IP (VoIP)
 - Online gaming

In summary:

TCP is used when **reliability and correctness** are important, while UDP is used when **speed and low delay** are more important than perfect reliability.

Diagram: TCP vs UDP at Transport Layer**(c) How TLS protects data privacy and integrity**

TLS (Transport Layer Security) is a security protocol that works above the transport layer (usually over TCP) to protect data during transmission.

TLS provides the following protections:

1. Confidentiality (Privacy):

- Data is **encrypted** using symmetric encryption algorithms.
- Even if an attacker intercepts the data, it appears as **unreadable ciphertext**.

2. Integrity:

- TLS uses **Message Authentication Codes (MAC)** or **hash functions** to ensure data is not altered.
- If data is modified in transit, the receiver can **detect the change** and discard the message.

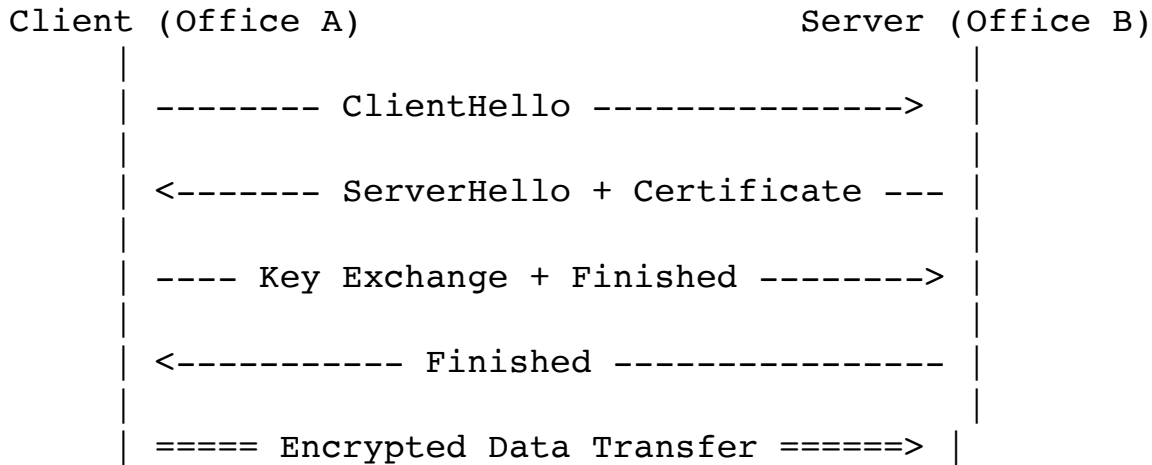
3. Authentication:

- TLS uses **digital certificates** to verify the identity of the server (and optionally the client).
- This prevents **man-in-the-middle attacks**.

4. Secure Key Exchange:

- During the **TLS handshake**, secure methods are used to **exchange encryption keys** safely before data transfer begins.

Diagram: TLS Handshake and Secure Data Transfer



Conclusion:

TLS ensures that data sent between two offices remains **private, unmodified, and trustworthy**, making it essential for protecting sensitive financial information over long-distance networks.

5.

Given

- Distance between A and B = **20,000 km** = 2×10^7 meters
- Transmission rate $R = 2$ Mbps = 2×10^6 bits/sec
- Propagation speed $s = 2.5 \times 10^8$ meters/sec
- File size = **800,000 bits**

First, compute **propagation delay**:

$$d_{prop} = \frac{\text{distance}}{\text{propagation speed}} = \frac{2 \times 10^7}{2.5 \times 10^8} = 0.08 \text{ seconds}$$

(a) Bandwidth-delay product and bits in the link

$$\text{Bandwidth-delay product} = R \times d_{prop} = (2 \times 10^6) \times 0.08 = 1.6 \times 10^5 \text{ bits} = 160,000 \text{ bits}$$

So, the **bandwidth-delay product** = **160,000 bits**.

Interpretation for the file transfer:

If the file is sent continuously as one large message, the **maximum number of bits that can be in the link at any time** is equal to the bandwidth-delay product.

160,000 bits

(b) Interpretation of bandwidth-delay product and width of a bit

Interpretation

The **bandwidth-delay product** represents the **maximum number of bits that can be “in flight” in the link at the same time** (i.e., bits that have been sent but not yet received).

So here, at most **160,000 bits** can be present in the link simultaneously.

Width of a bit

First, find how many bits fit in the link:

$$\text{Bits in link} = 160,000$$

$$\text{Total link length} = 2 \times 10^7 \text{ meters}$$

$$\text{Width of one bit} = \frac{\text{Link length}}{\text{Number of bits in link}} = \frac{2 \times 10^7}{1.6 \times 10^5} = 125 \text{ meters}$$

So, **width of one bit = 125 meters**.

A football field is roughly **100 meters** long.

Since **125 m > 100 m**, yes, **one bit is longer than a football field**.

(c) General expression for the width of a bit

Let:

- s = propagation speed (m/s)
- R = transmission rate (bits/s)
- m = length of the link (meters)

Propagation delay:

$$d_{prop} = \frac{m}{s}$$

Bandwidth-delay product:

$$R \times d_{prop} = R \times \frac{m}{s}$$

This is the **number of bits in the link**.

So, **width of one bit**:

$$\text{Bit width} = \frac{\text{Link length}}{\text{Number of bits in link}} = \frac{m}{\frac{Rm}{s}} = \frac{s}{R}$$

$$\text{Width of a bit} = \frac{s}{R}$$

✓ Final Answers Summary

- (a) Bandwidth-delay product = **160,000 bits**
Max bits in link at any time = **160,000 bits**
- (b) It represents **bits in flight**.
Width of one bit = **125 meters** → Yes, longer than a football field
- (c) General expression:

$$\text{Bit width} = \frac{s}{R}$$

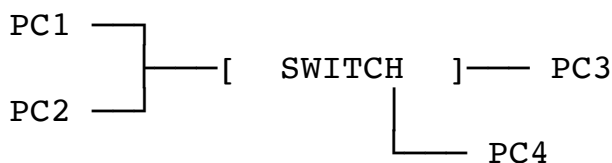
6. (a) What is a switch? Factors for selecting the right switching technique

A **switch** is a networking device that operates mainly at the **Data Link layer (Layer 2)** and is used to **connect multiple devices** within a LAN. It forwards data frames to the correct destination device using **MAC addresses**, which improves efficiency and reduces unnecessary traffic compared to a hub.

Functions of a switch:

- Learns MAC addresses of connected devices
- Forwards frames only to the intended port
- Reduces collisions and improves network performance

Diagram: Switch in a LAN



Factors responsible for selecting the right switching technique:

- **Network size and traffic load:** Large, busy networks need faster and more efficient switching.
- **Delay and performance requirements:** Real-time applications need low latency.
- **Reliability and fault tolerance:** Some techniques provide better reliability.
- **Cost and complexity:** Advanced techniques are more expensive and complex.
- **Type of data/application:** Voice, video, and data traffic may need different handling.

(b) Compare and contrast HTTP and HTTPS and how HTTPS helps in blocking issues

HTTP (HyperText Transfer Protocol):

- Data is sent in **plain text**.
- **No security**: Data can be read or modified by attackers.
- Uses **port 80**.
- Vulnerable to **eavesdropping, data tampering, and man-in-the-middle attacks**.

HTTPS (HTTP Secure):

- Uses **HTTP over TLS/SSL**.
- Data is **encrypted** before transmission.
- Uses **port 443**.
- Provides **confidentiality, integrity, and authentication**.

How HTTPS helps in blocking issues:

- **Encryption** blocks attackers from reading sensitive data (passwords, banking info).
- **Integrity checks** prevent data from being modified in transit.
- **Authentication using certificates** prevents fake websites and man-in-the-middle attacks.

Diagram: HTTP vs HTTPS

HTTP:

Client ----- Plain Text Data -----> Server
(Can be read/modified by attacker)

HTTPS:

Client ---[Encrypt via TLS]==> Secure Data ==> Server
<===[Decrypt via TLS]---
(Attacker sees only encrypted data)

(c) How SSL/TLS creates a set of security parameters

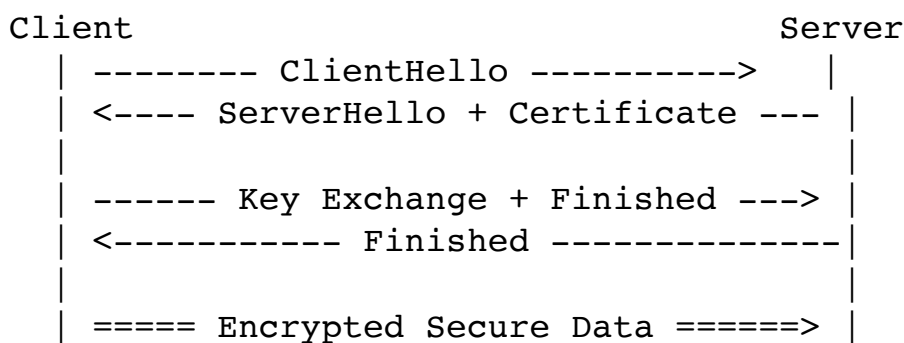
SSL/TLS creates security parameters during the **handshake process** before actual data transfer begins. These parameters define **how the communication will be secured**.

Steps involved:

1. **ClientHello:** Client sends supported encryption algorithms and TLS version.
2. **ServerHello:** Server selects the algorithms and sends its **digital certificate**.
3. **Authentication:** Client verifies the server's certificate.
4. **Key Exchange:** Both sides securely generate a **shared secret key**.
5. **Session Parameters Set:**
 - Encryption algorithm (e.g., AES)
 - Integrity algorithm (MAC/Hash)
 - Session keys
6. **Secure Communication Starts:** All data is now **encrypted and protected**.

These chosen algorithms and keys together form the **security parameters** for the session.

Diagram: SSL/TLS Handshake and Parameter Setup



In short:

SSL/TLS sets up **encryption method, integrity check method, authentication details, and secret keys** during the handshake, and then uses them to protect all transmitted data.

✅ Given

7.

- File size = 1.5 MB = $1.5 \times 1024 \times 1024$ bytes $\approx 1,572,864$ bytes
- In bits = $1,572,864 \times 8 = 12,582,912$ bits
- Packet size = 1 KB data = 1024 bytes = 8192 bits
- RTT = 80 ms = 0.08 s
- Handshaking time = $2 \times \text{RTT} = 160 \text{ ms} = 0.16 \text{ s}$
- Bandwidth = 10 Mbps = 10×10^6 bits/s

Number of packets:

$$\text{Packets} = \frac{1,572,864}{1024} = 1536 \text{ packets}$$

Transmission time per packet:

$$t_{tx} = \frac{8192}{10 \times 10^6} = 0.0008192 \text{ s} \approx 0.00082 \text{ s}$$

✅ (a) Continuous sending, bandwidth = 10 Mbps

Total data transmission time:

$$T_{data} = \frac{12,582,912}{10 \times 10^6} = 1.2582912 \text{ s} \approx 1.26 \text{ s}$$

Add handshaking time:

$$T_{total} = 0.16 + 1.26 = 1.42 \text{ s (approx)}$$

✅ Answer (a):

$T \approx 1.42 \text{ seconds}$

✅ (b) Must wait 1 RTT after each packet

Here, for each packet:

- Send packet (transmission time $\approx 0.00082 \text{ s}$)
- Then wait 1 RTT = 0.08 s

So, time per packet:

$$T_{per \text{ packet}} \approx 0.08 + 0.00082 \approx 0.08082 \text{ s}$$

For 1536 packets:

$$T_{data} = 1536 \times 0.08082 \approx 124.1 \text{ s}$$

Add handshaking:

$$T_{total} = 124.1 + 0.16 \approx 124.26 \text{ s}$$

✅ Answer (b):

$T \approx 124.3 \text{ seconds}$

✓ (c) Infinitely fast transmit, but only 20 packets per RTT

- In 1 RTT (0.08 s), we can send 20 packets
- Total packets = 1536

Number of RTTs needed:

$$\text{RTTs} = \frac{1536}{20} = 76.8 \Rightarrow 77 \text{ RTTs}$$

Time for data:

$$T_{data} = 77 \times 0.08 = 6.16 \text{ s}$$

Add handshaking:

$$T_{total} = 6.16 + 0.16 = 6.32 \text{ s}$$

✓ Answer (c):

$$T \approx 6.32 \text{ seconds}$$

✓ Final Answers (Write in exam):

- (a) $\approx 1.42 \text{ s}$
- (b) $\approx 124.3 \text{ s}$
- (c) $\approx 6.32 \text{ s}$

📌 Concept Diagram (for understanding / if asked about pipelining vs stop-and-wait)

(a) Continuous:

[Handshake] ==== Data Stream =====>

(b) Stop-and-Wait:

[H] -> P -> wait RTT -> P -> wait RTT -> P -> wait RTT -> ...

(c) Window-based (20 per RTT):

[H] -> 20 packets -> RTT -> 20 packets -> RTT -> ...

8. IoT Network Congestion and Data Flow Management

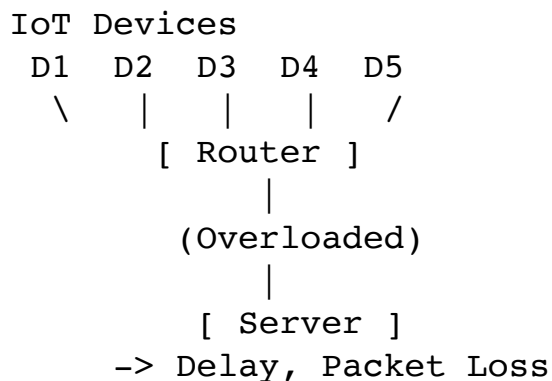
(a) What do you mean by congestion?

Congestion in a computer network occurs when the **demand for network resources exceeds the available capacity** of the network. When too many devices send data at the same time, routers and links become overloaded, causing:

- **Increased delay**
- **Packet loss**
- **Reduced throughput**
- **Unstable performance**

In an IoT system with many devices sending data to a central server, congestion can occur at **access links, routers, or the server side** due to heavy traffic.

Diagram: Congestion in a Network



(b) Techniques and algorithms to manage data flow while maintaining integrity and reliability

To control congestion and ensure **reliable and correct data delivery**, the following techniques and algorithms are used:

1. Flow Control

- Ensures the sender does not send data **faster than the receiver can handle**.
- Example: **Sliding Window Protocol** (used in TCP).
- Prevents receiver buffer overflow.

2. Congestion Control (TCP-based)

- Adjusts the sending rate based on network condition.
- Important algorithms:

- **Slow Start:** Starts with a small rate and increases gradually.
- **Congestion Avoidance:** Increases rate slowly to avoid overload.
- **Fast Retransmit & Fast Recovery:** Quickly retransmits lost packets and recovers without restarting from zero.
- Helps **reduce packet loss and delay** during congestion.

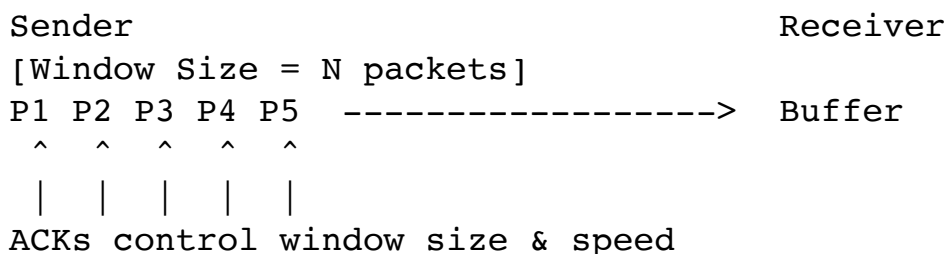
3. Error Control (Reliability & Integrity)

- **Error detection** using Checksum/CRC.
- **Retransmission (ARQ)** if packets are lost or corrupted.
- Ensures **data integrity and correctness**.

4. Traffic Shaping and Queue Management

- **Leaky Bucket / Token Bucket** algorithms control the rate of outgoing traffic.
- **Queue management** in routers (e.g., FIFO, priority queues) helps manage overload situations.

Diagram: Flow & Congestion Control (Sliding Window Idea)



In short:

Using **flow control, congestion control, and error control**, the network can manage heavy IoT traffic while maintaining **data integrity and reliability**.

(c) Role of multiplexing and demultiplexing

Multiplexing

Multiplexing is the process of **combining data from multiple devices or applications** and sending it over a **single shared network link**.

Benefits:

- Improves **efficient utilization of bandwidth**

- Allows **many IoT devices** to share the same connection
- Reduces **cost and resource usage**
- Increases **overall network efficiency**

Demultiplexing

Demultiplexing is the reverse process at the receiver side, where the received data is **separated and delivered to the correct application or device** using **port numbers or identifiers**.

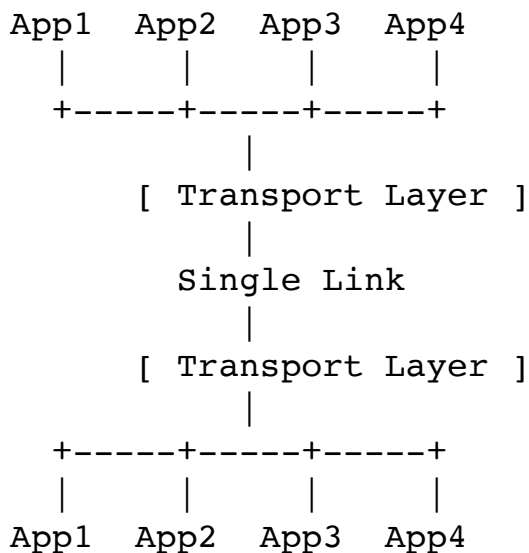
Role:

- Ensures **data goes to the correct process/device**
- Prevents **mixing of data streams**
- Maintains **accurate and reliable delivery**

Diagram: Multiplexing and Demultiplexing

Sender Side (Multiplexing)
(Demultiplexing)

Receiver Side



In summary:

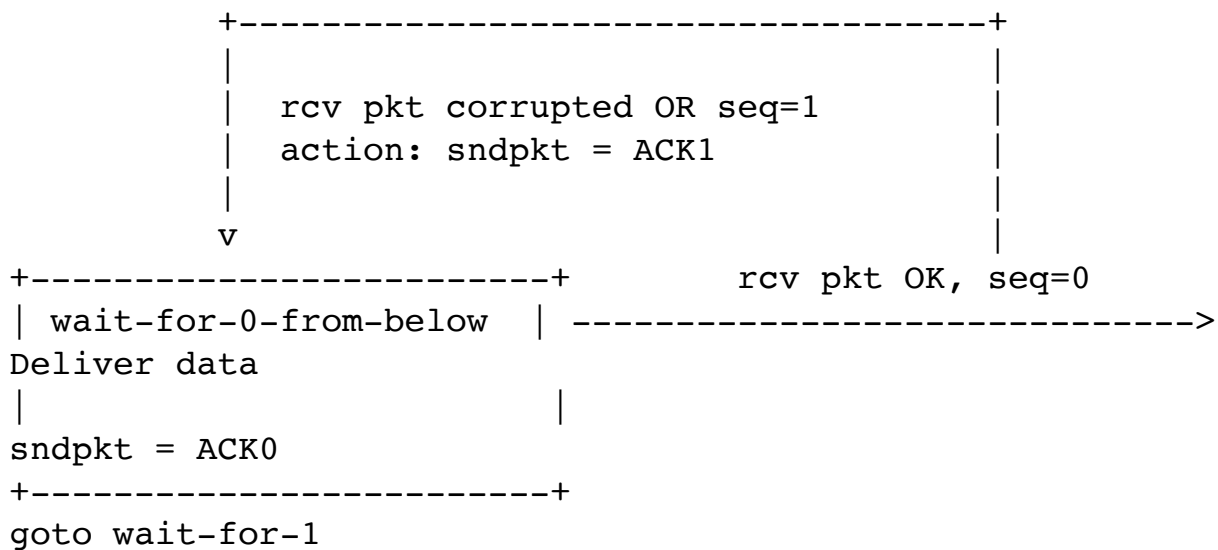
- **Multiplexing** combines multiple data streams to use the network **efficiently**.
- **Demultiplexing** separates them at the receiver to ensure **correct and accurate delivery**.

9. rdt 2.2 Receiver: Self-Transitions and ACK Handling

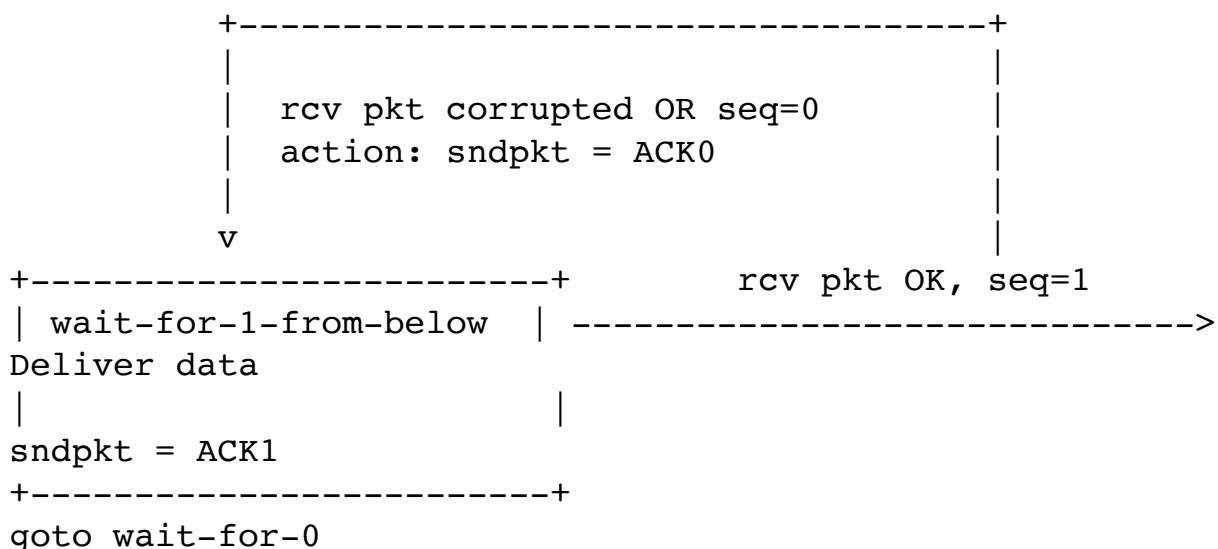
In **rdt 2.2**, the receiver uses **sequence numbers (0 and 1)** and sends **ACKs**. If a packet is **corrupted** or **duplicate**, the receiver **re-sends the last correct ACK** using a **self-transition** (i.e., it stays in the same state and sends an ACK again).

Receiver FSM (Simplified, Exam-Drawn Version)

State: wait-for-0-from-below



State: wait-for-1-from-below



These **self-transitions** send the **last correct ACK** again when a bad/duplicate packet arrives.

(a) If we remove the self-transition action in wait-for-1-from-below

Answer: ✗ The protocol would **NOT** work correctly.

Reason:

In `wait-for-1-from-below`, the receiver is **expecting packet with seq = 1**. If it instead receives:

- A **corrupted packet**, or
- A **duplicate packet with seq = 0**,

then it must **re-send ACK0** (the last correct ACK). This tells the sender: *"I already got 0 correctly; send me 1."*

If we **remove** this action:

- The receiver will **stay silent** on errors/duplicates.
- The sender, waiting for an ACK, may **timeout unnecessarily** or **get confused** about what to retransmit.
- This can cause **extra delays** or **incorrect behavior**.

So, the **self-transition ACK is necessary** for correctness and efficiency.

(b) If we remove the self-transition action in `wait-for-0-from-below`

Answer: ✗ Again, the protocol would **NOT** work correctly.

Reason:

In `wait-for-0-from-below`, the receiver expects **seq = 0**. If it receives:

- A **corrupted packet**, or
- A **duplicate packet with seq = 1**,

it must **re-send ACK1** (the last correct ACK) to tell the sender: *"I already got 1; now send 0."*

If this action is removed:

- The sender may **not get any feedback** about what the receiver has correctly received.
- This can lead to **unnecessary retransmissions, timeouts, and inefficient communication**.
- In some cases, progress can **stall**.

So, this self-transition is also essential.

(c) What do you mean by IP Spoofing?

IP Spoofing is a security attack in which an attacker **forges the source IP address** of packets to make them appear as if they come from a **trusted or legitimate host**.

Why attackers do this:

- To **bypass security filters or firewalls**
- To **impersonate** another machine
- To perform **Denial of Service (DoS)** or **Man-in-the-Middle** attacks

Simple explanation:

The attacker **lies about their IP address** in packets to **trick the receiver** into trusting or accepting the data.

Small Diagram: IP Spoofing Idea

```
Attacker (Fake IP = Trusted Host)
    |
    | Packet with forged source IP
    v
  [ Server ]
  "Thinks it's from a trusted host"
```

10. 📌 Network Setup (Diagram you can draw in exam)

```
A ----(Link 1)---- [Switch 1] ----(Link 2)---- [Switch 2]
----(Link 3)---- B
           d1, s1, R1           d (proc)           d2, s2, R2           d
(proc)           d3, s3, R3
```

Where:

- Packet length = **L bits**
- Link **i** has: length **di**, propagation speed **si**, transmission rate **Ri**
- Each switch has **processing delay = d**
- No queuing delay

(a) Total end-to-end delay (General Expression)

For **store-and-forward** packet switching, the packet must be **fully transmitted** on each link before moving to the next.

Total delay =

- **Transmission delays** on 3 links
- **Propagation delays** on 3 links
- **Processing delays** at 2 switches

$$D_{total} = \sum_{i=1}^3 \frac{L}{R_i} + \sum_{i=1}^3 \frac{d_i}{s_i} + 2d$$

That is:

$$D_{total} = \left(\frac{L}{R_1} + \frac{L}{R_2} + \frac{L}{R_3} \right) + \left(\frac{d_1}{s_1} + \frac{d_2}{s_2} + \frac{d_3}{s_3} \right) + 2d$$

(b) Given Values

- Packet size = **1500 bytes** = $1500 \times 8 = 12000$ bits
- Propagation speed (all links):

$$s = 2.5 \times 10^8 \text{ m/s}$$

- Transmission rate (all links):

$$R = 2 \text{ Mbps} = 2 \times 10^6 \text{ bits/s}$$

- Switch processing delay:

$$d = 3 \text{ ms} = 0.003 \text{ s}$$

- Link lengths:

- $d_1 = 5000 \text{ km} = 5 \times 10^6 \text{ m}$
- $d_2 = 4000 \text{ km} = 4 \times 10^6 \text{ m}$
- $d_3 = 1000 \text{ km} = 1 \times 10^6 \text{ m}$

(c) Numerical End-to-End Delay Calculation

✓ 1. Transmission delay per link:

$$\frac{L}{R} = \frac{12000}{2 \times 10^6} = 0.006 \text{ s} = 6 \text{ ms}$$

For 3 links:

$$3 \times 6 \text{ ms} = 18 \text{ ms}$$

✓ 2. Propagation delays:

$$d_{prop1} = \frac{5 \times 10^6}{2.5 \times 10^8} = 0.02 \text{ s} = 20 \text{ ms}$$

$$d_{prop2} = \frac{4 \times 10^6}{2.5 \times 10^8} = 0.016 \text{ s} = 16 \text{ ms}$$

$$d_{prop3} = \frac{1 \times 10^6}{2.5 \times 10^8} = 0.004 \text{ s} = 4 \text{ ms}$$

Total propagation delay:

$$20 + 16 + 4 = 40 \text{ ms}$$

✓ 3. Processing delays at 2 switches:

$$2d = 2 \times 3 \text{ ms} = 6 \text{ ms}$$

✓ 4. Total End-to-End Delay:

$$D_{total} = 18 \text{ ms} + 40 \text{ ms} + 6 \text{ ms} = 64 \text{ ms}$$

$D_{total} = 64 \text{ ms}$

✓ Final Answers (Write in Exam)

(a)

$D_{total} = \sum_{i=1}^3 \frac{L}{R_i} + \sum_{i=1}^3 \frac{d_i}{s_i} + 2d$
--

(c)

$D_{total} = 64 \text{ ms}$
